

Contents

1 - Mint (create the wallet) 2

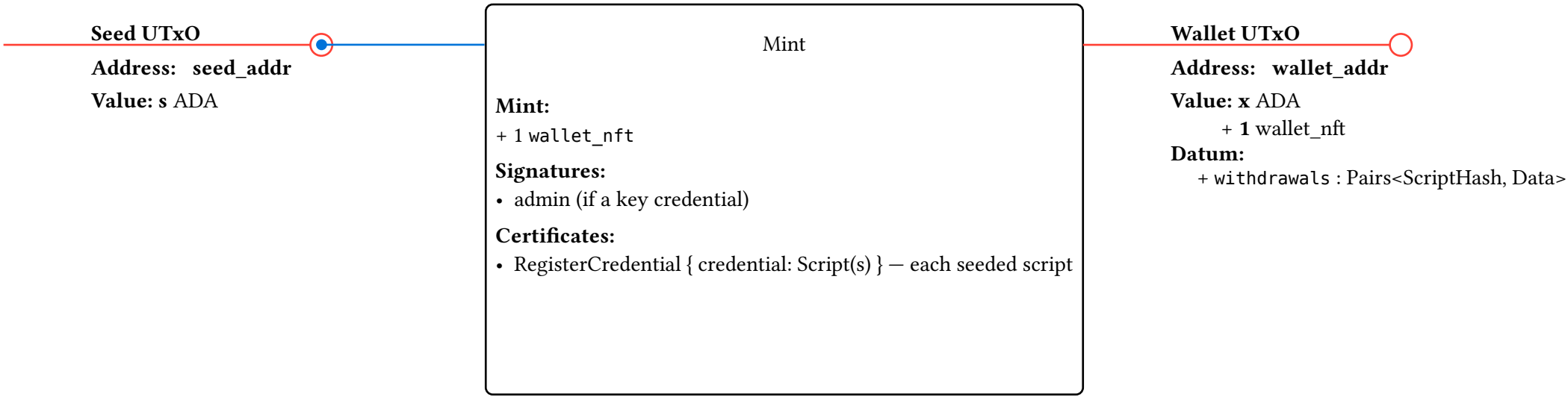
2 - Spend (base M-of-N + delegated authorizations) 3

3 - Update withdrawals (per-UTxO) 4

4 - Close (admin burns the NFT) 5

1 - Mint (create the wallet)

The wallet is created by minting its identifying NFT — one token under the script’s own policy — into a fresh wallet UTxO at the wallet address. The admin must authorize the mint.



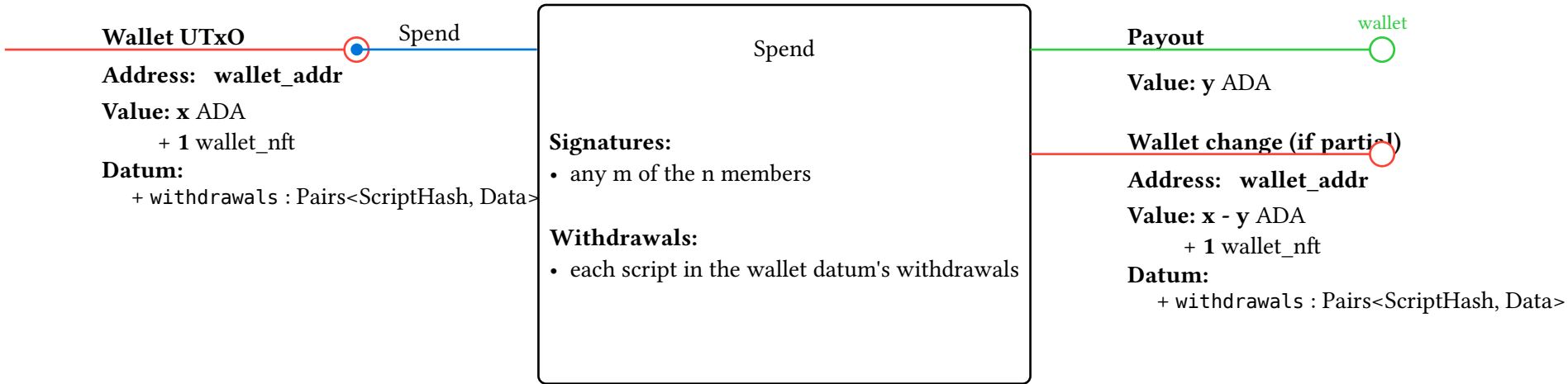
Notes:

- wallet_addr: payment is wallet_hash, staking any. wallet_hash = hash(smart_wallet(seed_utxo, settings_ref, wallet_nft_name, admin)) — the multivalidator is parameterized by its seed, configuration reference, NFT name, and the admin credential, so all wallet UTxOs share one address and one NFT policy.
- one-shot: the mint requires the seed_utxo to be spent, and a UTxO can be spent at most once, so this policy id mints exactly one token ever. The resulting NFT is unforgeable and identifies the wallet.
- the mint redeemer is Mint { out_ix }, locating the new wallet UTxO.
- the new wallet UTxO carries the NFT, an inline datum with a withdrawals map, and no reference script. Every script in that map must be registered in this same transaction — a RegisterCredential certificate for its stake credential, which runs the script’s publish handler. The handler reads its own initial Data from this output’s datum and validates it, so a wallet may be created already carrying delegated restrictions, but only ones that self-validated their initial state.
- the admin credential (a script parameter) must be satisfied: a key admin signs; a script admin authorizes by a withdraw-0 invocation.

Figure 1: Mint (create the wallet)

2 - Spend (base M-of-N + delegated authorizations)

The wallet pays out. The spending script enforces its base M-of-N signature floor and forwards every other check to the withdrawal scripts named in the wallet UTxO's datum; the single spend redeemer asserts their conjunction.



- Notes:**
- `wallet_addr`: payment is `wallet_hash`, staking any. A wallet UTxO holds its identifying NFT plus ada and arbitrary native assets.
 - `wallet_hash` = `hash(smart_wallet(seed_utxo, settings_ref, wallet_nft_name, admin))`: the spending script is parameterized by its seed, configuration reference, NFT name, and the `admin` credential, so all wallet UTxOs share one address. Externalizing the members/threshold config lets it change without redeploying the script.
 - validity: the spend requires the wallet UTxO to carry its NFT (`1 wallet_nft` under `wallet_hash`) — only NFT-bearing UTxOs are valid smart wallets.
 - base M-of-N floor: the wallet's configuration supplies members and threshold, and the spend requires at least threshold of members present in `extra_signatories`. A member may itself be a script credential, so the floor can be another multisig, a DAO, and so on. This design is agnostic of how that configuration is sourced.
 - delegated authorizations: every `ScriptHash` key in the wallet UTxO's `withdrawals` datum field is a `withdraw-0` staking script that must run in this transaction; the `Data` value carries that script's own mutable state. The Spend redeemer asserts the conjunction — the M-of-N floor **and** all delegated checks — so no restriction can be silently omitted. The map is per-UTxO, so different wallet UTxOs may carry different restriction sets.
 - the delegated scripts are the extension point: spending limits, time locks, recipient allow-lists, oracle attestations, ... each is an independent withdrawal script observed here, none known to the spending script.
 - composability: the spend asserts only its own inputs and that the required withdrawals ran — never the total input or output counts, nor unrelated UTxOs. Any partial change keeps the NFT so the wallet stays valid.

Figure 2: Spend (base M-of-N + delegated authorizations)

3 - Update withdrawals (per-UTxO)

An authorized admin credential — a parameter of the spending script — rewrites a wallet UTxO’s delegated `withdrawals` list in place. Funds are untouched: same address, same value — only the restriction set changes.

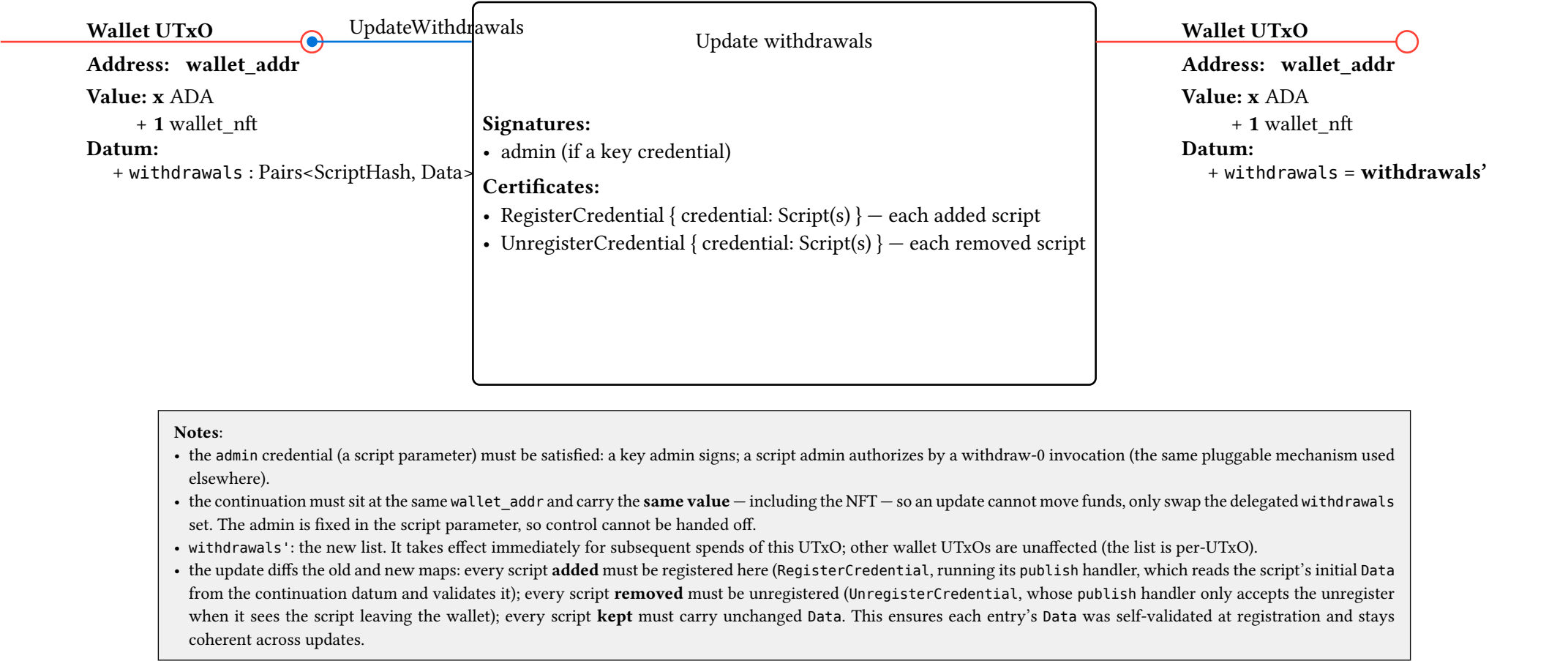
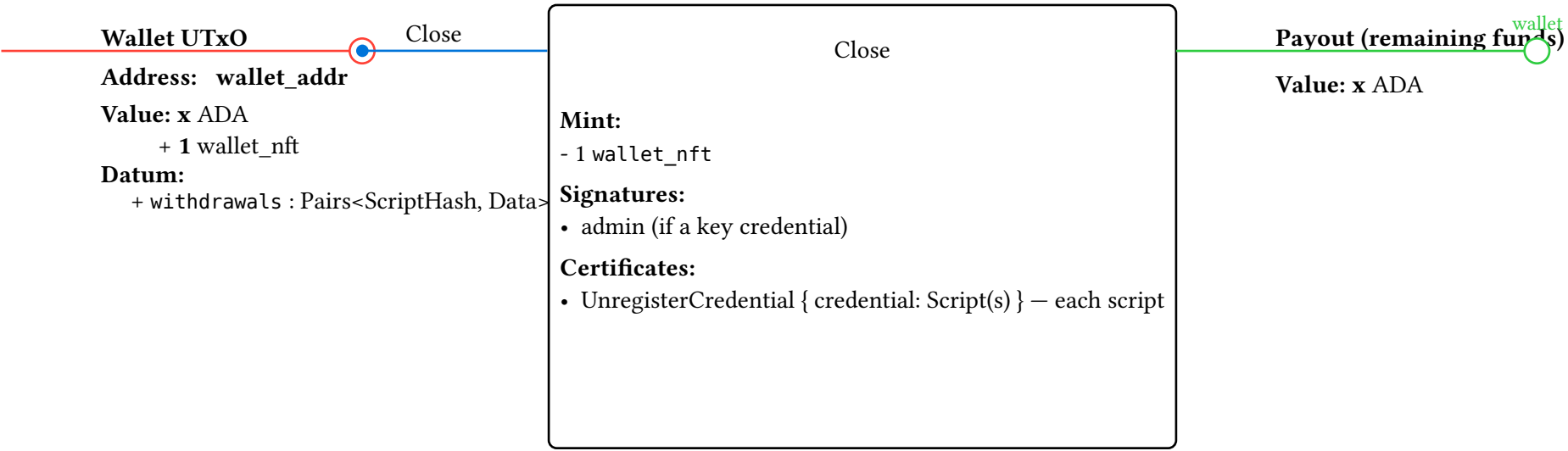


Figure 3: Update withdrawals (per-UTxO)

4 - Close (admin burns the NFT)

The admin closes the wallet: the wallet UTxO is spent, releasing whatever funds remain, and the identifying NFT is burned. The Close spend redeemer requires the admin, the burn, and that every delegated withdrawal script is unregistered so its stake deposit is refunded; the Burn mint redeemer permits the burn.



- Notes:**
- the admin credential (a script parameter) must be satisfied: a key admin signs; a script admin authorizes by a withdraw-0 invocation.
 - the wallet NFT — 1 wallet_nft under wallet_hash — must be burned in this transaction (-1), destroying the wallet's identity. The burn runs the mint endpoint with a Burn redeemer, which only checks the burn quantity; authorization is enforced by the Close spend redeemer.
 - closing spends the wallet UTxO and releases any remaining funds. There is no continuation: once the NFT is gone, the wallet can no longer be spent.
 - every script in the closing wallet's withdrawals map must be unregistered here (UnregisterCredential), refunding its stake deposit. Each script's publish handler accepts this because it sees itself in the wallet input and no wallet output.

Figure 4: Close (admin burns the NFT)